Public



Rostislav Markov is principal engineer with Amazon Web Services, Inc. in New York and a long-term contractor with Merck & Co, Inc. His work lies at the intersection of clinical data processing, health AI, and modern engineering practices.

Public



Specification-driven governance for AI-assisted code generation

Public

Plausible code is not the same as compliant code

Thesis
For regulated SDTM pipelines, safer AI-assisted coding starts with better specifications, not just better models.

Regulated risk
Incorrect transformations can create rework, traceability gaps, validation burden, submission risk.

Loose prompts fail
Prose leaves the model to infer the algorithm, schema, edge cases, and framework patterns.

Case-study opportunity
In this case, BDD specs, tests, metadata constraints, and conventions existed.

Research question
How does increasing prompt governance affect the correctness and integration quality of generated transformation functions?

Public

Experiment design: three prompt levels, eight production functions

Prompt levels

A1 Function description
Brief prose only

A2 Behavioral specification
Gherkin BDD scenarios with examples

A3 Fully governed spec
BDD + conventions + pytest + types

Evaluation sample (Python)

Low	get_first_dose, get_last_dose
Medium	get_last_contact, call_patient_death, call_subjid
High	call_suppqual, create_epoch_taetord, create_se_domain

Scoring: 10 binary functional checks + 3 integration criteria scored 0-3 each.

Public

What was measured: correctness vs. integration

Functional correctness (F%)

Does the function solve the right problem?

- ✓ Correct algorithm
- ✓ Correct join keys
- ✓ All target vars mapped
- ✓ Validation rules & exact errors
- ✓ Edge cases; no side effects

Integration quality (I%)

Does the function fit the project?

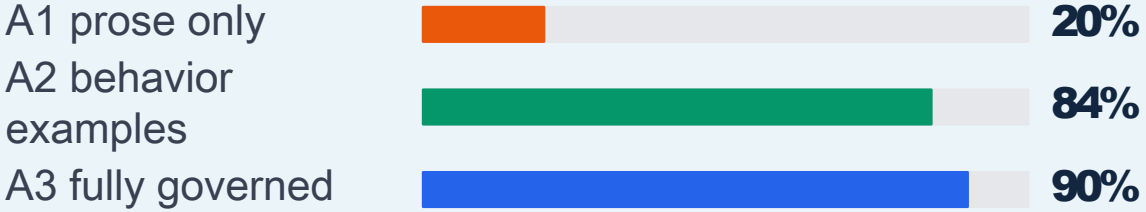
- ✓ Imports, decorator, configuration
- ✓ Signature and structure
- ✓ Test pass rate

Key distinction: BDD governs what to do; conventions and tests govern how to fit.

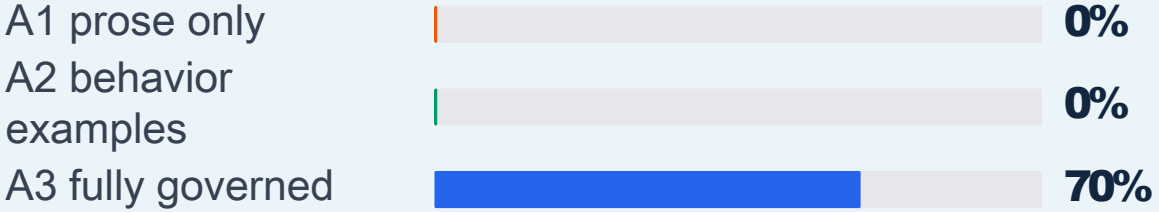
Public

Main result: behavior examples fix correctness; conventions fix integration

Functional correctness



Integration quality



+64 points from BDD-style examples

The biggest single gain came from concrete input/output scenarios that removed ambiguity from the derivation.

+70 points from conventions + tests

BDD expressed what to derive, but not which imports, type contracts, decorators, helpers, or return structure to use.

Lesson

Specification quality changes the failure mode: from semantic errors (wrong algorithm) to mechanical errors (wrong import path).

Public

Loose prompts caused overengineering and wrong assumptions

Dose derivations

Simple production functions were generated as large modules with treatment-period logic, PK half-life extension, and extra flags.

Last contact

A straightforward maximum-date derivation became a large follow-up status engine with custom enums and dataclasses.

SUBJID propagation

The model built an ID construction function instead of propagating an existing SUBJID from the DC domain.

What fixed the behavior?

Concrete examples: named SDTM variables, explicit input/output rows, multiple subjects, edge cases, and expected error messages.

Operational lesson

If the behavior is only described in prose, the model fills gaps with generic clinical-programming assumptions.

Governance lesson

Make the desired behavior concrete before asking the model to produce code.

Public

Takeaway: govern the prompt with SDLC artifacts

BDD governs what the function does.	Conventions and tests govern how it fits.
Correctness 20% → 90%	Integration 0% → 70%

Takeaway
Invest first in specification quality before model selection or fine-tuning.

Public



Thank you.

Public

Detailed results by function

Function	A1 F%	A1 I%	A2 F%	A2 I%	A3 F%	A3 I%
get_first_dose	10%	0%	90%	0%	100%	89%
get_last_dose	10%	0%	80%	0%	100%	100%
get_last_contact	20%	0%	90%	0%	90%	78%
call_patient_death	40%	0%	90%	0%	90%	67%
call_subjid	20%	0%	80%	0%	90%	56%
call_suppqual	30%	0%	80%	0%	80%	56%
create_epoch_taetord	30%	0%	80%	0%	80%	56%
create_se_domain	0%	0%	80%	0%	80%	56%
Average	**20%**	**0%**	**84%**	**0%**	**90%**	**70%**

Public



Rostislav Markov
Amazon Web Services, Inc.
12 W 39th St
New York, NY 10018
rmarkov@amazon.com
rosmarkov.github.io